Anand Bhaskaran. Work Samples.

Three case studies on how I think and ship

For Orderfox · Senior AI & Data Product Engineer · 2026

CASE 1 · KNOWLEDGE GRAPH AS AGENT FOUNDATION

AI Brain. A knowledge-graph context layer that grounds production agents in business intelligence.

LumApps · 2026 · Architect & builder

PROBLEM Eight-plus data silos: CRM activity, conversation transcripts, usage analytics, AE notes. Agents drowning in raw text, hallucinating relationships ("who owns this account?", "what did the customer ask last quarter?"). Vector search returned plausible chunks but lost the relational structure that actually drives business decisions.

DECISION Build a knowledge graph (Neo4j + Graphiti + Zep) instead of expanding the vector store. Reasoning: the business questions agents need to answer are relational, not semantic. "Which accounts have a stalled deal AND a recent product complaint AND no exec sponsor?" is a graph query, not a similarity search. Embeddings sit on top; the graph is the ground truth.

APPROACH

- Defined the ontology: Account, Contact, Conversation, Insight, Initiative, Signal. Typed relationships (works_at, mentioned_in, blocks, depends_on).
- LLM extractors turn unstructured text into typed triples; Graphiti handles temporal versioning so old beliefs do not poison new context.
- Agents query the graph through a thin tool layer, not raw Cypher. Each tool returns structured rows with provenance back to the source.
- Telemetry (Langfuse) on every agent turn: which graph queries fired, which returned empty, where the agent fell back to vector search. That signal drives the next round of ontology extensions.

8+ silos	Production	C-level
UNIFIED INTO ONE GRAPH	POWERS AI AGENTS COMPANY-WIDE	DIRECT REPORTING ON OUTCOMES

Neo4j · Graphiti · Zep · Python · LangChain · FastAPI · Langfuse · GCP

LESSON *Ontology beats embeddings when the question is "who and why," not "what is similar." Start with the relationships the product needs to reason over, then layer vectors on top for the fuzzy edges.*

CASE 2 · BUILD VS BUY, DECIDED IN NUMBERS

LLM translation pipeline. $337K vendor contract eliminated in six weeks.

Beekeeper · 2023 to 2024 · Solo build, then scaled across product teams

PROBLEM External vendor translating ~100 i18n key-value strings per week. $337K annual contract. Two-week turnaround blocked product iteration; localized features always shipped a sprint behind English. Product team treated translation as a non-negotiable tax.

DECISION Build, not buy. Translation quality on structured product strings is now an LLM commodity; the vendor's real cost was lock-in and latency, not labor. Risk-adjusted: a six-week build cost ~10% of one year's vendor spend, with the upside of perpetual ownership.

APPROACH

- GitHub Actions trigger on any i18n PR. Diff detection so we only translate changed keys.
- OpenAI batch API with glossary injection (product nouns, brand terms, tone constraints) per locale.
- Confidence scoring: high-confidence translations auto-merge, ambiguous strings (idioms, UI-context-dependent) get flagged for human review in a lightweight web UI.
- Six-week breakdown: week 1 prototype on one locale, week 2 glossary + tone, week 3 reviewer UX, weeks 4 to 5 staged rollout across 14 locales, week 6 vendor contract not renewed.

$337K	12K+	6 weeks	Same day
ANNUAL COST ELIMINATED	ASSETS TRANSLATED SINCE JAN 2024	PROTOTYPE TO PRODUCTION	I18N TURNAROUND (VS 2 WEEKS)

Python · OpenAI · GitHub Actions · Vue (review UI) · PostgreSQL

> **LESSON** *"Good enough" beats "perfect on paper" when the alternative is no shipping. The vendor was technically pristine but operationally slow; the LLM pipeline was technically imperfect but unblocked the product team forever.*

CASE 3 · AGENTS THAT ACT, NOT ACT AUTONOMOUSLY

AI outbound agent. 2x open rates, 2x approved opportunities, two to three hours per rep per day.

LumApps · 2026 · Built solo, rolled out across SDR and AE teams

PROBLEM SDR and AE teams sending identical templated outbound. 8 to 12% open rates. Two to three hours per rep per day on drafting and research. Sales leadership wanted "more outbound" but adding headcount linearised the problem, not solved it.

DECISION Human-in-the-loop drafting, NOT autonomous sending. Reasoning: in B2B outbound, one bad send loses an account permanently. The 10% efficiency gain from full automation is not worth the 1% catastrophic risk. The agent's job is to do the research and propose the draft; the rep keeps the send button.

APPROACH

- Signal ingestion: CRM (account state, last touchpoint), recent news, LinkedIn activity, product usage (where relevant). Each signal typed and timestamped so the agent can reason about freshness.
- Agent generates three drafts per prospect with explicit reasoning ("opening on X because Y was mentioned in their Q3 earnings call"). Reps see the reasoning, not just the draft.
- Tool design: each agent has one to two tools max. Tested in Langfuse with replay-driven evals; new prompts gated on regression against a golden set of 200 historical outbound conversations.
- Telemetry loop: open rates, reply rates, meeting-booked rates, by prompt variant. Bad variants killed within a week.

2x	2x	2 to 3 hrs	$1M/qtr
OPEN RATES	APPROVED OPPORTUNITIES	SAVED PER REP PER DAY	FORECAST REVENUE AT 25% CONVERSION

Python LangChain OpenAI FastAPI Langfuse Salesforce GCP

LESSON *Agents that act do not have to act autonomously. The valuable action is the reasoning, the research, the draft. Keeping a human on the send button costs you nothing measurable and buys you trust the system cannot earn back if it misfires once.*

Public write-up of the outbound agent build: I Built an AI Outbound Agent. Here's What Actually Worked. · TowardsAI, May 2025.
Follow-up on human-in-the-loop drafting: Drafted, not sent. · The Compounding Curiosity, May 2025.

Contact: anandbaskaran3193@gmail.com · +41 76 304 3193 · Zurich